Data Collection

> [!IMPORTANT]
>
> - The factory image already includes preset datasets required for training traffic sign recognition and lane detection models. This tutorial is provided for users who need to train their own YOLOv11 models and want to collect their own datasets
> - If you want to experience the complete functionality directly, you can skip this section!!!

1. Course Content

1. Master the data collection methods required for training YOLOv11 object detection models. Training YOLOv11 object models requires first preparing datasets containing the objects to be detected

2. Traffic Sign Recognition Model Data Collection

2.1 Principle Explanation

- Record videos of targets to be detected under different lighting conditions and backgrounds through the camera, then obtain the original image dataset by capturing video frames at fixed intervals

2.2 Data Collection

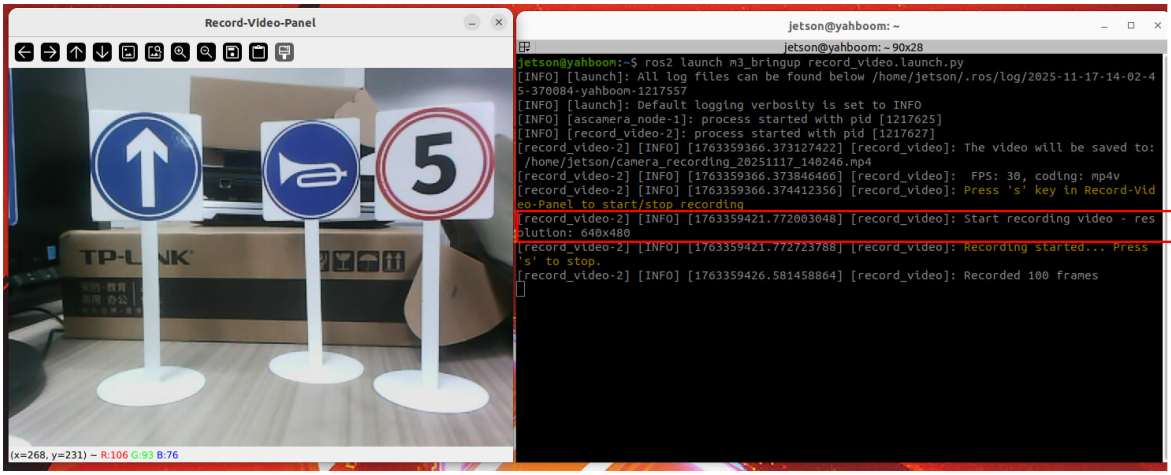
- Launch the data collection recording node

```
#orin nano、rdkx5
ros2 launch auto_driver record_video.launch.py
```

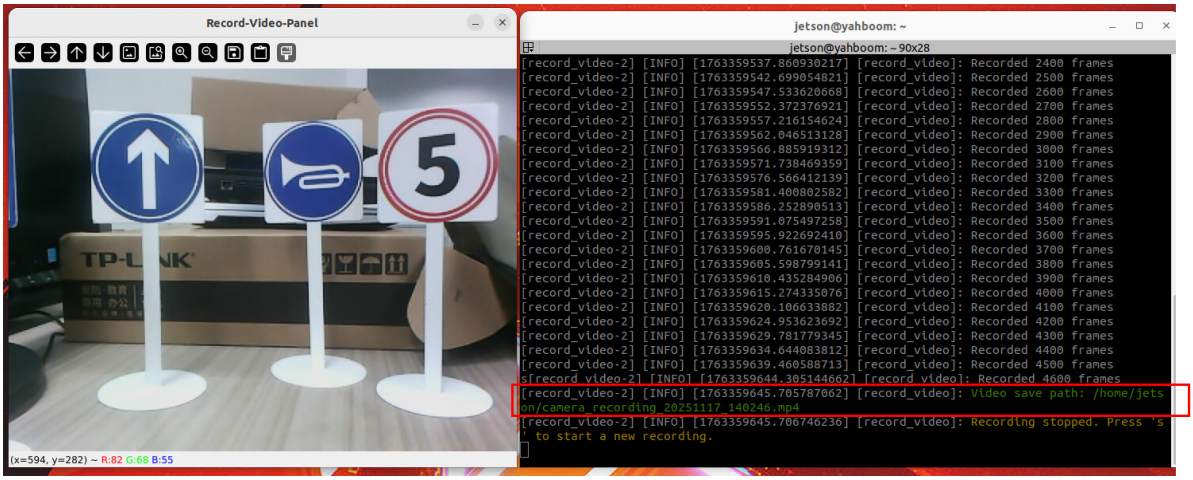
Optional Launch Parameters

- **output_path:** Path to save video files
- **fps:** Frame rate of recorded video

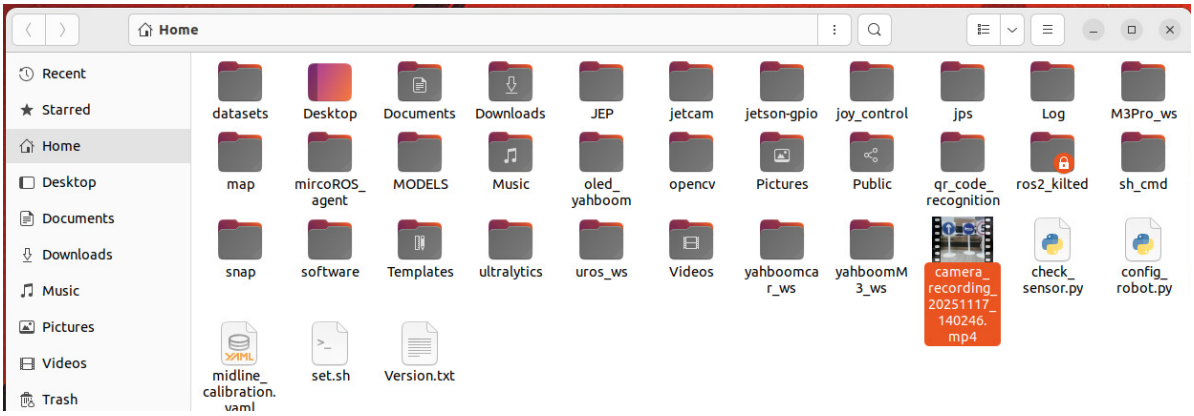
- After running the program, a video window will open. You can record video materials containing targets to be detected from different angles and under different lighting conditions by moving the chassis, which can enrich the diversity of the dataset.
- Place the traffic signs to be recognized within the view and start recording video



- Press Ctrl+C to end recording, and the terminal will print the path of the saved file



- If no video output path is specified, video files will be saved in the system user's home directory by default



- Split video frames into image sets. Place split_video.py from the "Model Training" folder in the source code summary to the root directory, then run the following command

```
cd ~
```

- Modify output_dir and video_file to your own image output path and video path

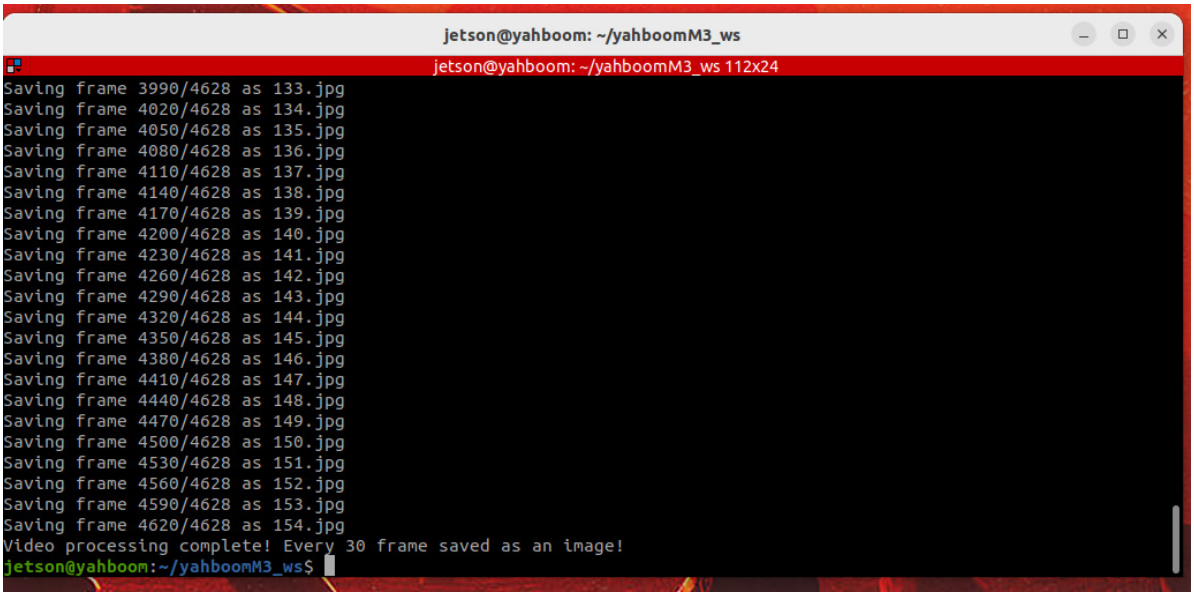
```
python3 split_video.py --output_dir ~/output_frame  --video_file ~/camera_recording_20251117_140246.mp4
```

Parameter Description

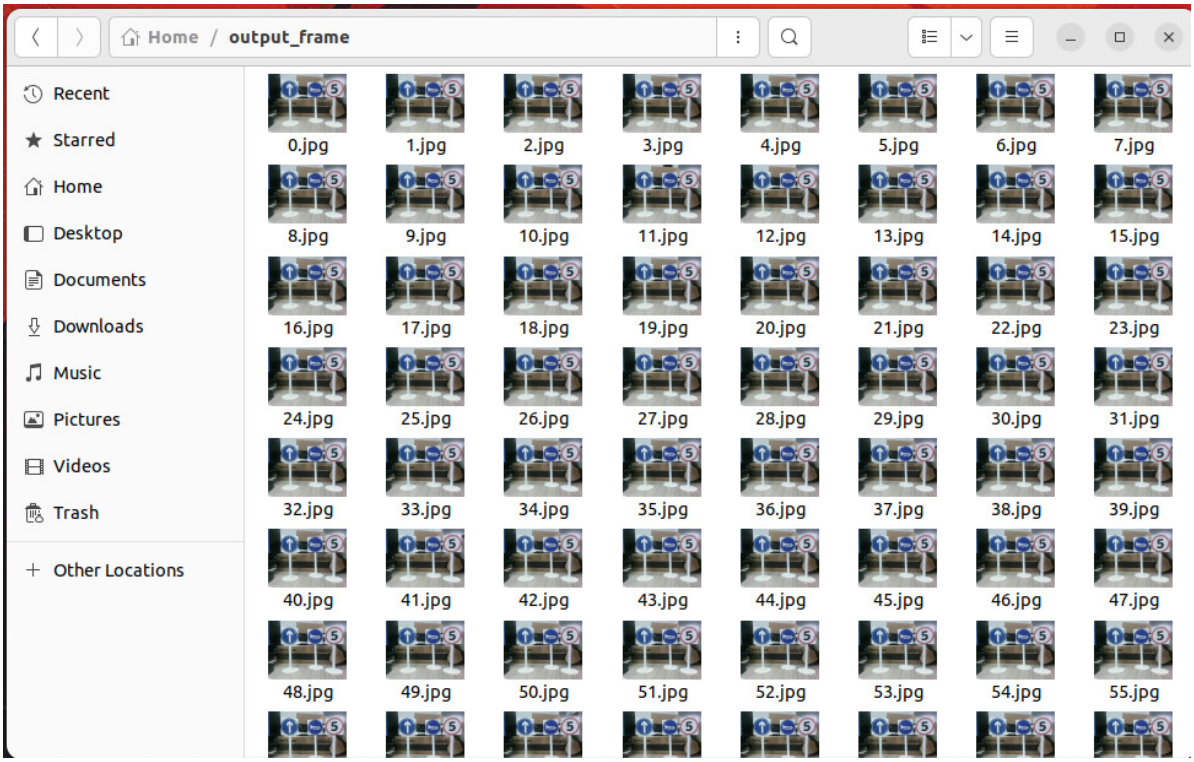
- **video_file:** Path of the video to be split
- **output_dir:** Path to save the split images
- **start_number:** Starting number for images
- **frame_interval:** Video frame interval, specifying how many frames to skip before capturing an image

---

- After splitting video frames is completed, the terminal will print prompt information



- The split images will be saved in the folder specified by output_dir



3. Lane Detection Model Dataset Collection

The lane detection model data collection process is the same as the traffic sign recognition model data collection process

3.1 Data Collection

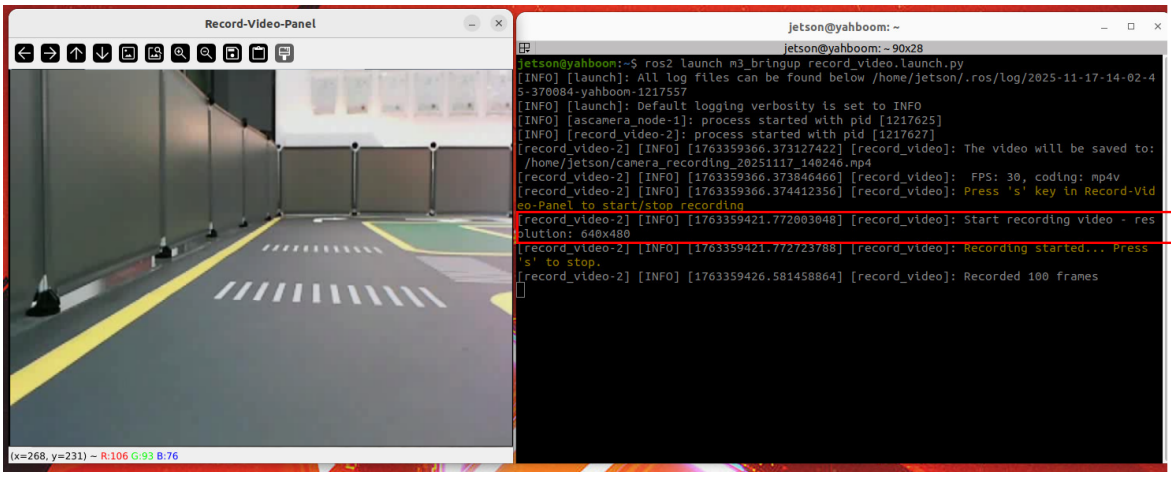
- Launch the data collection recording node

```
ros2 launch auto_driver record_video.launch.py
```

Optional Launch Parameters

- **output_path:** Path to save video files
- **fps:** Frame rate of recorded video

- Place the mouse over the Record-Video-panel window and press the 's' key to start recording video



- The video saving and splitting methods are completely consistent with the previous ones, so they will not be repeated here.

4. Source Code Explanation

4.1 Recording Video from Video Topic

- The source code is located in record_video.py in the auto_drive package, source code path:

```
#orin nano 、rdkx5
~/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/auto_drive/record_video.py
```

- The image callback function (image_callback method) is triggered every time an image message is received, responsible for:
  Format conversion: Convert ROS image messages (msg) to OpenCV format (cv_image, bgr8 encoding, corresponding to RGB color space) through self.bridge.imgmsg_to_cv2.
  Display image: Use cv2.imshow to pop up a window named Record-Video-Panel to display images in real-time.
  Key control: Detect key presses through cv2.waitKey(1) (1ms delay, ensuring window refresh):
  Press 's' key: Toggle recording state (start if not recording, stop if recording).

Recording logic: If self.is_recording is True, write the current frame to video through self.out.write(cv_image) and count (print log every 100 frames).
Exception handling: Catch errors in conversion or recording, prompt with red log.

```
def image_callback(self, msg):
    try:
        cv_image = self.bridge.imgmsg_to_cv2(msg, desired_encoding='bgr8')

        cv2.imshow("Record-Video-Panel", cv_image)
        key = cv2.waitKey(1) & 0xFF


        if key == ord('s'):
            if not self.is_recording:
                # Start recording
                self.start_recording(cv_image)
            else:
                # Stop recording
                self.stop_recording()

        # If recording is in progress, write the frame.
        if self.is_recording and self.out is not None:
            self.out.write(cv_image)
            self.frame_count += 1
            if self.frame_count % 100 == 0:
                self.get_logger().info(f"Recorded {self.frame_count} frames")

    except Exception as e:
        self.get_logger().info(Fore.RED+f"Error processing image:
{str(e)}"+Fore.RESET)
```

- Recording control methods
  start_recording method (start recording)
  Initialize cv2.VideoWriter:
  Get the width and height of the first frame (first_frame.shape[:2], OpenCV format is (height, width)).
  Parse encoding format with cv2.VideoWriter_fourcc (fourcc_code, e.g., mp4v corresponds to *'mp4v').
  Concatenate video save path, create VideoWriter instance (parameters: path, encoding, frame rate, width and height).
  Validate initialization: If VideoWriter failed to open (e.g., path error or unsupported encoding), print red error log.
  Update status: self.is_recording = True, and print green log prompt for recording start.
  stop_recording method (stop recording)
  Update status: self.is_recording = False.
  Release resources: If self.out exists (i.e., recording is in progress), call self.out.release() to close the video file, ensuring data is written completely.
  Reset count: self.frame_count = 0, self.out = None (re-initialize for next recording).
  Print logs: video save path (green) and re-recording prompt (yellow).

```
    def start_recording(self, first_frame):
        """Start recording video"""
        if self.out is None:
```

```
            self.frame_height, self.frame_width = first_frame.shape[:2]
            fourcc = cv2.VideoWriter_fourcc(*self.fourcc_code)
            video_path = os.path.join(self.output_path, self.video_name)
            self.out = cv2.VideoWriter(
                video_path,
                fourcc,
                self.fps,
                (self.frame_width, self.frame_height)
            )

            if not self.out.isOpened():
                self.get_logger().error(Fore.RED+"Unable to open video editor.
Please check the encoding format and output path."+Fore.RESET)
                return

            self.get_logger().info(Fore.GREEN+f"Start recording video -
resolution: {self.frame_width}x{self.frame_height}"+Fore.RESET)

        self.is_recording = True
        self.get_logger().info(Fore.YELLOW+"Recording started... Press 's' to
stop."+Fore.RESET)

    def stop_recording(self):
        """Stop recording video"""
        self.is_recording = False
        if self.out is not None:
            self.out.release()
            self.get_logger().info(Fore.GREEN+f"Video save path:
{os.path.join(self.output_path, self.video_name)}"+Fore.RESET)

            self.out = None
            self.frame_count = 0

        self.get_logger().info(Fore.YELLOW+"Recording stopped. Press 's' to
start a new recording."+Fore.RESET)
```

4.2 Launch Program

- Used to start camera driver and the node for recording video from camera image topic in 4.1
- Source code is located in record_video.launch.py in the auto_drive package, source code path:

```
#orin nano 、rdkx5
~/yahboomcar_ros2_ws/yahboomcar_ws/src/auto_drive/launch/record_video.launch.py
```

- Read camera type from environment variable CAMERA_TYPE:
  If it's nuwa: Start hp60c.launch.py camera driver, subscribe to image topic /ascamera_hp60c/camera_publisher/rgb0/image.
- Node startup logic
  First start the appropriate camera driver (bringup_camera_launch) to ensure image stream is published normally.
  Then start the video recording node (record_video_node), passing parameters: save path, frame rate, and corresponding camera's image topic.

The recording node (corresponding to the previous ImageToVideoNode) will subscribe to the specified image topic and execute recording function.

```
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node
from launch.launch_description_sources import PythonLaunchDescriptionSource
from launch.actions import IncludeLaunchDescription,TimerAction
from launch.actions import DeclareLaunchArgument
from launch.substitutions import LaunchConfiguration

def generate_launch_description():
    CAMERA_TYPE = os.environ['CAMERA_TYPE']#Get camera type
    m3_bringup_dir=os.path.join(get_package_share_directory('auto_drive'))
    if CAMERA_TYPE == 'nuwa':
        bringup_camera_launch = IncludeLaunchDescription(
            PythonLaunchDescriptionSource([
                os.path.join(get_package_share_directory('ascamera'), 'launch'),
                '/hp60c.launch.py'
            ])
        )

    elif CAMERA_TYPE == 'usb':
        bringup_camera_launch = IncludeLaunchDescription(
            PythonLaunchDescriptionSource([
                os.path.join(get_package_share_directory('usb_cam'), 'launch'),
                '/camera.launch.py'
            ])
        )


    #Startup parameters
    declared_arguments = []
    declared_arguments.append(
        DeclareLaunchArgument(
            "output_path",
            default_value="~",
            description="Video save path/视频保存路径 ",
        )
    )
    declared_arguments.append(
        DeclareLaunchArgument(
            "fps",
            default_value=30,
            description="Video frame rate/视频帧率 ",
        )
    )


    output_path=LaunchConfiguration('output_path')
    fps = LaunchConfiguration('fps')


    record_video_ndoe=Node(
        package='auto_drive',
        executable='record_video',
```

```
        name='record_video',
        output='screen',
        parameters=[
            {'output_path': output_path},
            {'fps': fps},
            {'image_topic': rgb_topic},
        ],
    )
    return LaunchDescription([
        *declared_arguments,    #Declare startup parameters
        bringup_camera_launch,  #Start the camera driver
        record_video_ndoe,       #Start recording video node
    ])
```

4.3 Video to Image Set Splitting Program

- Source code path:

```
~/Rosmaster/Sample/split_video.py
```

- Core process (main function)
  Initialize configuration:
  Read command line arguments to get video path, output folder and other configurations.
  Automatically create output folder (if it doesn't exist).
  Video processing:
  Use cv2.VideoCapture to open video file, get total frames, frame rate and other information.
  Loop through video frames, save 1 image every frame_interval frames (e.g., every 30 frames):
  Image naming starts from start_number and increments sequentially (e.g., 0.jpg, 1.jpg).
  Print progress in real-time (current frame / total frames + saved image name).

```
import cv2
import os
import argparse
def get_args():
    parser = argparse.ArgumentParser()
    parser.add_argument('--output_dir', default='engine', help='Output folder',
type=str)
    parser.add_argument('--frame_interval',default=30 ,help='Capture an image at
specified intervals from the video frames', type=int)
    parser.add_argument('--video_file', default='', help='Original video path',
type=str)
    parser.add_argument('--start_number', default=0, help='Image starting
number', type=int)
    return parser.parse_args()

if __name__ == "__main__":
    args = get_args()

    # Path to the input video file
```

```
    video_file =args.video_file
    # Path to the output folder to save the frames
    output_folder =args.output_dir
    # Check if the output folder exists, if not, create it
    if not os.path.exists(output_folder):
        os.makedirs(output_folder)
    # Open the video file using OpenCV
    cap = cv2.VideoCapture(video_file)
    # Get the total number of frames in the video
    total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
    # Get the frames per second (fps) of the video
    fps = cap.get(cv2.CAP_PROP_FPS)
    # Frame counter to keep track of the current frame number
    frame_number = 0
    # Define the interval for extracting frames (every 15th frame)
    frame_interval = args.frame_interval
    # Initialize a new counter for saved frames (starting from 1)
    start_number = args.start_number
    while True:
        ret, frame = cap.read()
        # If the frame is not successfully read, exit the loop
        if not ret:
            break
        # Save every 15th frame
        if frame_number % frame_interval == 0:
            # Format the frame number starting from 1 (e.g., 1.png, 2.png)
            image_name = f'{start_number}.jpg'
            image_path = os.path.join(output_folder, image_name)
            # Save the current frame as a PNG image
            cv2.imwrite(image_path, frame)
            print(f'Saving frame {frame_number}/{total_frames} as {image_name}')
            # Increment the saved frame counter
            start_number += 1
        # Increment the frame counter for the video
        frame_number += 1


    # Release the video capture object
    cap.release()
    # Print message when processing is complete
    print(f"Video processing complete! Every {frame_interval} frame saved as an
image!")
```